Presence

heartbeats, expiring keys, careful fan-out

Watch the 4-min video

In one line: clients send periodic **heartbeats** that refresh a **key with a TTL**, so a user goes offline automatically when the beats stop; the expensive part is **fan-out**, which you limit to friends who are online and looking, debounce, and replace with **pull** for large groups.

1 The busiest green dot

A chat app with 100 million users shows a green dot next to friends who are online. People connect, lose signal, close laptops and switch apps every second, and each change might be interesting to hundreds of friends. A rough sum shows the danger: if 50 million users are connected and each changes state about once every 10 minutes, that is ~83,000 changes a second; with 200 friends each, a naive design sends **~17 million presence events a second**, more than all the chat messages combined. Presence is a small feature with a big write and fan-out load.

2 Detecting online and offline

Online is easy: the WebSocket opened (lesson 26). **Offline** is hard, because the most common way to leave is not a clean logout: the phone enters a tunnel, the OS suspends the app, the laptop lid closes. TCP may not notice a dead peer for minutes. So the client sends a tiny **heartbeat** every ~30 s (a WebSocket ping frame or a small app message) and the server treats silence as absence.

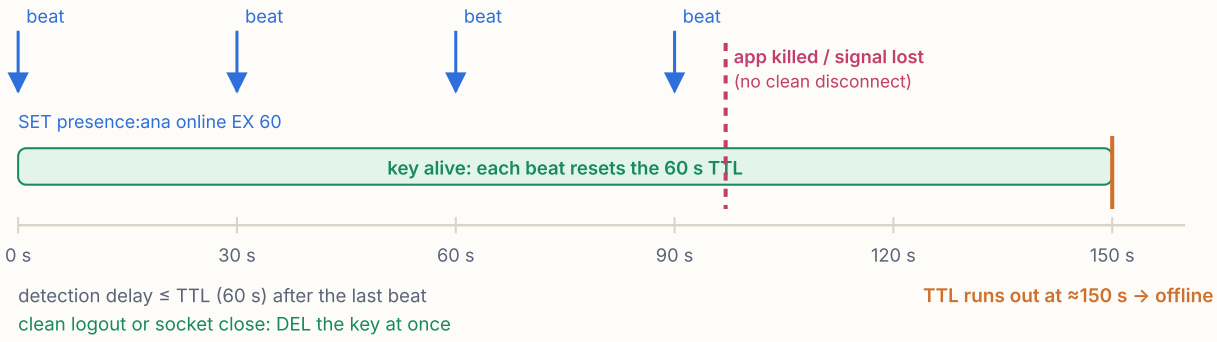


Figure 1. Each heartbeat resets the key's TTL. After the last beat, the key survives at most one TTL, then the user is offline with no cleanup job.

The interval is a trade-off: a 10 s beat detects absence faster but triples the traffic and drains phone batteries (each beat can wake the radio). Set the TTL to about **2× the interval** so one lost beat does not flip the user offline. A clean disconnect (socket close event on the gateway, or logout) deletes the key immediately, so the TTL only matters for crashes and lost signal.

3 Storing presence with expiring keys

In Redis each beat runs `SET presence:42 online EX 60` (or `EXPIRE` on an existing key). While beats arrive, the key lives; when they stop, it disappears. Crashed clients and crashed gateways cannot leave a stale green dot forever. Some details that matter at scale:

- **Shard by user ID.** Presence writes are tiny but constant: 50M users / 30 s ≈ 1.7M writes a second, spread across a cluster.
- **Let gateways batch.** The gateway already sees every beat; it can refresh keys in pipelined batches, or own presence for its sockets and write only on change, using the TTL as a safety net if the gateway itself dies.
- **Expiry events are not instant.** Redis deletes expired keys lazily (on access) and by background sampling, and keyspace notifications fire only when the key is actually removed. Use the TTL for truth, but don't promise friends an offline event to the second.
- **Keep it small.** Store the online flag (and maybe status and device) separately from profile data, so the hottest key is a few bytes.

4 Last seen, statuses and privacy

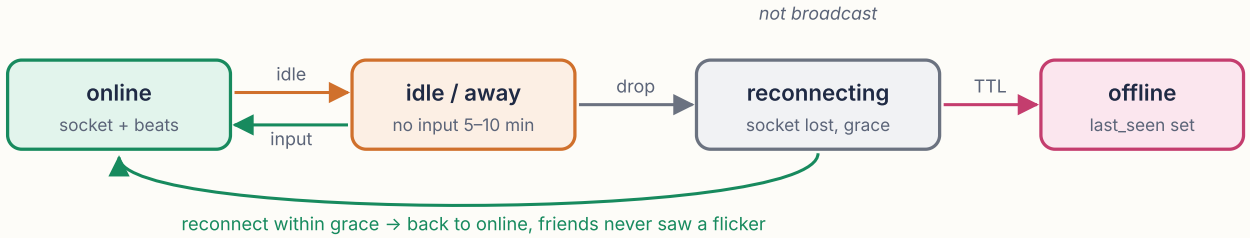


Figure 2. A practical status model. The grace state absorbs flaps (bad signal, app switch) so friends do not see on-off-on.

Last seen is a timestamp of the last activity, kept with the presence data or in the user store. It powers "last seen 10 minutes ago". Update it at most once a minute, not on every beat, and write it durably when the user goes offline. A user with several devices is online if **any** device is; status is the most active one (online beats idle beats offline). Privacy settings ("hide last seen", "show to contacts only") must be applied in the presence service on read, before anything is sent to another user.

5 The fan-out problem and how to limit it

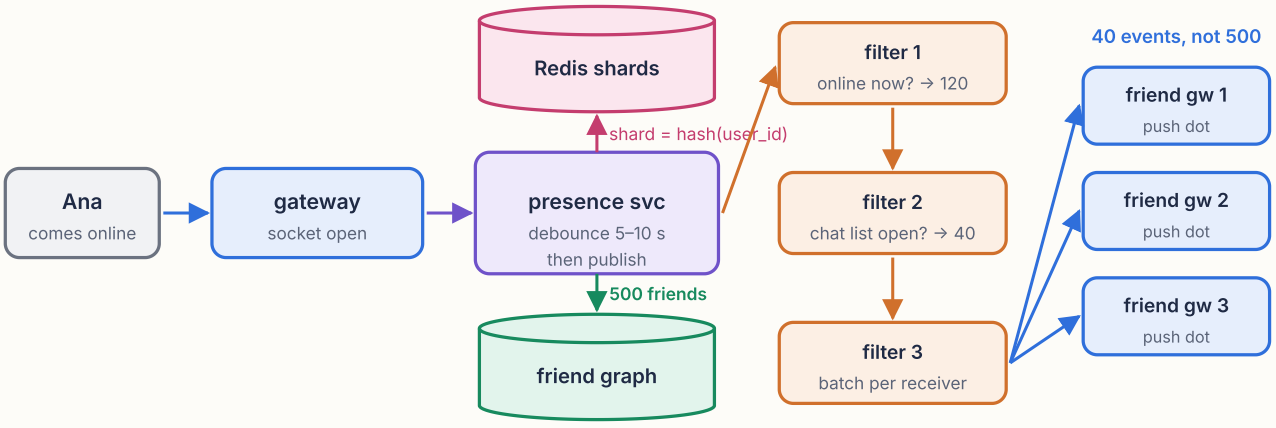


Figure 3. Ana comes online. Instead of 500 events, the presence service filters to friends who are online and have a view that shows her, after a short debounce.

Technique	What it does	Effect
Only online friends	intersect the friend list with the presence store	offline friends will fetch on open
Only interested viewers	clients subscribe to the presence of users visible on screen (open chat, contact list)	often cuts events by 10× or more
Debounce / flap suppression	announce offline only after a grace period (e.g. 10–30 s); ignore on-off-on	removes the noisiest events
Batching	group changes per receiver and send every few seconds	fewer, larger messages
Pull for large groups	no push; client fetches online members when it opens the list, refreshes periodically	cost grows with viewers, not members

For a channel with 50,000 members, pushing every status change to everyone is hopeless (each change × 50,000). Show a count ("1,204 online") from a cheap approximate counter, and load the visible part of the member list lazily as the user scrolls.

WORKED EXAMPLE: ONLINE TO FRIENDS

Ana's app beats every 30 s; each beat runs `SET presence:ana online EX 60` on her shard. When her socket opens, the presence service waits a few seconds (in case she drops straight away), then reads her 500 friends, keeps the ~120 that have a live presence key, and of those the ~40 that have subscribed to her (her chat is on their screen). Each of those gets a small `{user: ana, status: online}` event through their own gateway. When she closes the app without logging out, the beats stop, the key expires within a minute, `last_seen` is written, and those friends see "last seen just now". Total cost: about two tiny writes per user per minute plus a few dozen events per change, which scales horizontally by adding shards.

TRADE-OFFS

- Short heartbeat and TTL: accurate dots, more traffic and battery use. Long: cheaper, but ghosts stay green for longer.
- Push presence: live dots, cost grows with friends × changes. Pull: cheap, but slightly stale.
- Debounce hides flaps but delays real offline transitions by the grace period.
- Strong accuracy is not needed: presence is a hint, so prefer availability and low cost over consistency.

COMMON MISTAKES

- Relying only on logout or socket-close events: crashes and lost signal leave users online forever.
- Writing last_seen to the main database on every heartbeat.
- Broadcasting every change to every friend, including offline ones and huge groups.
- Ignoring privacy settings on the fan-out path, leaking a status the user hid.

6 Key terms

Heartbeat: small periodic message proving the client is still there

TTL key: key that expires unless refreshed; absence = offline

Grace period: delay before announcing offline, to absorb flaps

Last seen: timestamp of last activity, updated coarsely

Presence fan-out: sending a status change to interested users

Subscription: client asks for presence only of users on screen

7 Interview questions

Q1. How do you track online status for 100M users? BASIC

Clients send heartbeats that refresh expiring keys in a sharded store like Redis, so users go offline automatically when beats stop; keep a coarse last-seen timestamp alongside.

Q2. Why use a TTL rather than marking users offline on disconnect? BASIC

Most exits are not clean (lost signal, killed app, crashed gateway); a TTL makes silence mean offline without any cleanup job.

Q3. How do you stop presence updates from exploding? SCENARIO

Fan out only to friends who are online and currently looking, debounce flaps, batch updates per receiver, and let large groups pull presence.

Q4. How would you choose the heartbeat interval? SCENARIO

Balance detection time against traffic and battery: around 30 s with a TTL of about two intervals is common; tighten only where faster detection is worth the cost.

Q5. A user has the app open on a phone and a laptop and closes the laptop. What should friends see? SCENARIO

Still online: presence is per device and the user's status is the most active device; only when all devices expire does last_seen get set.

Q6. Can you rely on Redis keyspace notifications to announce 'offline' exactly when the TTL ends? TRICKY

Not precisely: expired keys are removed lazily or by background sampling, and the event fires on removal, so it can lag. It is fine for a hint, not for strict timing.

Q7. Why is presence usually eventually consistent? TRICKY

It is a hint that changes constantly; a dot a few seconds stale harms nobody, while strong consistency would multiply the write and fan-out cost.

Next: SD-Q-003 · Notification Systems backendroadmap.com